Graph Search Algorithms

Kanishk Goel

14 November 2025

Contents

1 Free Primitives

Treat any algorithm with linear running time as something we can essential use for free.

2 Generic Graph Search

Input: An undirected/directed graph $G = (V, E)$, and a starting vertex $s \in V$
Output: Identify the vertices of V reachable from s in G.

Generic Graph Search

Input: graph $G = (V, E)$ and vertex $s \in V$
Output: a vertex is reachable from s if and only if it is marked as "explored"

---

mark s as explored and all others unexplored
while $\exists\, (v, w) \in E$ *with* v *explored and* w *unexplored* **do**
  choose some edge (v, w)
  mark w as explored

3 Breadth First Search

Explores layer by layer, Layer 0 being s itself.

Breadth First Search (BFS)

---

mark s as explored, all other vertices as unexplored
$Q \leftarrow$ a queue data structure, init with s
while Q *is not empty* **do**
  remove the vertex from the front of Q, call it v
  foreach *edge* (v, w) *in* v*'s adjacency list* **do**
    if w *is unexplored* **then**
      mark w as explored
      add w to the end of queue

Runtime - $O(m + n)$

4 Connected Components

In an undirected graph $G = (V, E)$, a maximal subset $S \subseteq V$ of vertices such that there is a path from any vertex in S to any other vertex in S.
UCC algorithm is used which takes undirected graph $G = (V, E)$ in adjacency-list representation with $V = \{1, 2, 3, \ldots, n\}$
Postcondition: $\forall\, u, v \in V$, $cc(u) = cc(v) \iff u, v$ are in the same connected component

Undirected Connected Components (UCC)

```
mark all vertices as unexplored
numCC ← 0
for i ← 1 to n do
    if i is unexplored then
        numCC ← numCC + 1                          // new component
        // call BFS
        Q ← a queue data structure, init with s
        while Q is not empty do
            remove the vertex from the front of Q, call it v
            foreach edge (v, w) in v's adjacency list do
                if w is unexplored then
                    mark w as explored
                    add w to the end of queue
```

Runtime - $O(m+n)$

5 Depth First Search

Takes one path and only backtracks when absolutely necessary.

Iterative Implementation of DFS

```
mark all vertices as unexplored
S ← a stack data structure, init with s
while S is not empty do
    pop the vertex v from the front of S
    if v is unexplored then
        mark v as explored
        foreach edge (v, w) in v's adjacency list do
            push w to the front of S
```

Recursive Implementation of DFS

```
Func DFS(G, v):
    mark s as explored and all other as unexplored
    foreach edge (s, v) in s's adjacency list do
        if v is unexplored then
            DFS(G, v)
```

Runtime - $O(m+n)$

6 Topological Sort

Topological Ordering

Let $G = (V, E)$ be a directed graph. A topological ordering of G is an assignment $f(v)$ of every vertex $v \in V$ to a different number such that

$$\forall\, (v, w) \in E,\ f(v) < f(w)$$

- Topological orderings do not exist in directed cyclic graphs.
- Every DAG has a topological ordering.

Topological Sort

DAG $G = (V, E)$ in adjacency list representation is given a f value for every vertex constituting a topological ordering. It uses DFS-Topo which assigns every vertex reachable from a vertex s an f-value.

TopoSort

```
Func TopoSort(G):
    mark all vertices as unexplored
    curLabel ← |V|
    foreach v ∈ V do
        if v is unexplored then
            DFS-Topo(G, v)
```

DFS-Topo

```
Func DFS-Topo(G, s):
    mark s as explored
    foreach edge (s, v) in s's outgoing adjacency list do
        if v is unexplored then
            DFS-Topo(G, v)
    f(s) ← curLabel
    curLabel ← curLabel − 1
```

curLabel is a global variable, initialised to $|V|$. Runtime is $O(m+n)$.

7 Strongly Connected Components

A strongly connected component or SCC of a directed graph is a maximal subset $S \subseteq V$ of vertices such that there is a directed path from any vertex in S to any other vertex in S.
Every directed graph can be viewed as directed acyclic graph built up from its SCCs.

The SCC Meta-Graph is Directed Acyclic

Let $G = (V, E)$ be a directed graph. Define the corresponding meta-graph $H = (X, F)$ with one meta-vertex $x \in X$ per SCC of G and a meta-edge (x, y) in F whenever there is an edge in G from a vertex in the SCC corresponding to x to one in the SCC corresponding to y. Then H is a directed acyclic graph.

Therefore, it is basically a compressed graph where each SCC of the graph is compressed into a node.

7.1 Kosaraju

Corollary: For a directed graph G, let $f(v), g(v)$ denote position of $v \in V$ as computed by TopoSort algorithm for G and G^{rev} respectively. Also, let S_1, S_2 denote two $SCCs$ of G with an edge (v, w) with $v \in S_1$ and $w \in S_2$.

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

$$\min_{x \in S_1} g(x) > \min_{y \in S_2} g(y).$$

We can see that the vertex x with $f(x) = 1$ will be the source vertex i.e for two connecting $SCCs$ the one with x will be the source of the directed edge. Therefore, for G^{rev}, it corresponds to a vertex y such that $g(y) = 1$ and y resides in the 'sink' SCC. (Sink SCC is the SCC that connects to no other SCC). It is vital to identify these sink $SCCs$ for Kosaraju.

Kosaraju

Input: directed graph $G = (V, E)$ in adjacency-list representation, with $V = \{1, 2, 3, \ldots, n\}$

Output: for every $v, w \in V, scc(v) = scc(w) \iff v, w$ are in the same SCC of G

$G^{rev} \leftarrow G$ with all edges reversed
mark all vertices of G^{rev} as unexplored

`// computes` $f(v)'s$ `magical ordering`
TopoSort(G^{rev})

`// finds SCCs in reverse topological order`
mark all vertices of G as unexplored
$numSCC \leftarrow 0$
foreach $v \in V$ in increasing order *of* $f(v)$ **do**
  if v *is unexplored* **then**
    $numSCC \leftarrow numSCC + 1$
    DFS-SCC(G, v)

DFS-SCC

Input: directed graph $G = (V, E)$ in adjacency list representation, and a vertex $s \in V$.

Output: every vertex reachable from s is marked as "explored" and has an assigned scc-value.

---

mark s as explored
$scc(s) \leftarrow numSCC$
foreach edge (s, v) in s's outgoing adjacency list **do**
  if v is unexplored **then**
    DFS-SCC(G, v)

Runtime: $O(m + n)$ where $m = |E| \wedge n = |V|$

Obvious Optimization - run DFS on the original graph, just go backwards on the edges.